

33 УТВЕРЖДЕН

КМНТ.

00000-00

00

-0-0-ЛУ

СПО «TNParserRT»

Версия 1.5.24 от 20.03.26

КМНТ. 00000-00 00 -0-0

2026 г.

Содержание

1. Список принятых сокращений.....	4
2. Введение.....	5
3. Применение утилиты «TNParserRT».....	6
3.1. Обмен между платой сбора накопителя ТНЗ и компьютером.....	6
3.2. Настройка утилиты «TNParserRT» для захвата UDP-потока накопителя ТНЗ.....	6
3.3. Использование файлов записей ТНЗ в качестве файла задания «TNParserRT».....	9
3.4. Конвертирование файлов записей ТНЗ в файл задания «TNParserRT».....	9
4. Выходной формат выдачи данных утилиты «TNParserRT» в UDP.....	10
4.1. Формат маркерного UDP-пакета «INFO».....	10
4.2. Алгоритм формирования контрольной суммы маркерного пакета.....	11
4.3. Режим формирования выходного UDP-потока без маркерного пакета.....	12
4.4. Подмешивание номеров тиков ТНЗ в UDP-пакеты данных «DATA».....	12
5. Конфигурирование выходных потоков утилиты «TNParserRT».....	14
5.1. Нумерация кожухов, слотов, каналов и параметров в конфигурационном файле..	16
5.2. Информация о статусах кожухов ТНЗ.....	17
5.3. Конфигурирование выходных потоков для модулей АЦП.....	19
5.3.1 Модули АЦП/010 и АЦП/310.....	21
5.4. Конфигурирование выходных потоков для модулей ARINC-429.....	21
5.4.1 Режим "all".....	21
5.4.1 Периодический режим "raw".....	22
5.4.1 Периодический режим "real".....	22
5.4.1 JSON-секция "A429".....	24
5.5. Конфигурирование выходных потоков для модулей CAN/RS.....	24
5.6. Конфигурирование выходных потоков для модулей IRIG.....	25
5.7. Конфигурирование выходных потоков для модулей ТНЗ/CHC/001.....	26

5.8. Конфигурирование выходных потоков для модулей ТНЗ/МПК/301 и ТНЗ.ПК-1/2..	26
5.9. Конфигурирование выходных потоков для модулей ТНЗ/СЧ/301.....	26
5.10. Конфигурирование выдачи в базу данных SQLITE.....	27
5.11. Конфигурирование выдачи в базу данных PostgreSQL.....	28
6. ЧМИ приложение «tnparsergui».....	29
7. Утилита «tnreplay».....	30
7.1. Назначение.....	30
7.2. Возможности.....	30
7.3. Примеры применения.....	30
8. ЧМИ приложение «tnreplaygui».....	32
9. Русификация консольного вывода утилит «tnparserrrt» и «tnreplay».....	33

1. Список принятых сокращений.

ASCII	- American Standard Code for Information Interchange, стандарт кодирования символов;
GUI	- Graphical User Interface (то же, что и ЧМИ);
NMEA	- National Marine Electronics Association, формат сообщений СНС;
SDI	- Source/Destination Identifier — 2-битный код слова ARINC429, который идентифицирует источник или место назначения;
UDP	- User Datagram Protocol;
UTC	- Coordinated Universal Time (всемирное координированное время);
АЦП	- аналого-цифровой преобразователь;
БД	- база данных;
КП	- комплекс программ (программных средств);
ОС	- операционная система;
ПК	- персональный компьютер;
ПО	- программное обеспечение;
СНС	- спутниковая навигационная система;
СПО	- специальное программное обеспечение;
ТНЗ	- твердотельный накопитель (3-я версия);
ЦМР	- цена младшего разряда;
ЧМИ	- человеко-машинный интерфейс (то же, что и GUI).

2. Введение.

Специальное программное обеспечение (СПО) «TNParserRT» предназначено для преобразования полного Ethernet потока данных от твердотельного накопителя ТНЗ в режиме реального времени в отдельные потоки данных блоков, модулей, каналов или параметров для каждого потребителя.

Выделенные данные модулей могут быть направлены в легко разбираемом виде на заданные UDP-адреса и порты, либо сохранены в базы SQL.

Минимальные выделяемые блоки информации для различных типов модулей:

- измерение канала АЦП;
- слово (метка) линии ARINC-429;
- полный («сырой») поток байт от сетевых модулей, спутниковых модулей и модулей последовательных портов;
- NMEA-поток (ASCII) от спутниковых модулей;
- данные UDP-пакета (payload) для сетевых модулей МПК или ПК платы сбора;
- частота, период или длительность модулей СЧ/001 и РК/301;
- отдельные слова в миноре по смещению для модулей IRIG.

Для распределённой системы возможно выделение данных как отдельного удалённого (дочернего блока), так и отдельного модуля.

На один поток данных (одному потребителю) можно перенаправить данные от всех доступных модулей блоков распределённой системы, сконфигурированных для выдачи данных в режиме реального времени.

Данные в режиме реального времени высылаются платой сбора блока ТНЗ, без команд запроса данных от персонального компьютера после подачи питания и инициализации блока корректным заданием по включению тумблера «ЗАПИСЬ». Данные приходят от платы сбора порциями («тиками») через каждые 1/1024 секунды.

СПО «TNParserRT» принимает все пакеты, приходящие от накопителя, разбирает их на данные отдельных модулей и выдаёт потребителям при получении каждого тика ТНЗ. При этом утилита «TNParserRT» сохраняет последовательность данных, поступающих с ТНЗ, и гарантирует тактовую частоту выдачи с точностью, обусловленной конфигурацией ПК и операционной системы.

3. Применение утилиты «TNParserRT».

3.1. Обмен между платой сбора накопителя ТНЗ и компьютером.

Соединение и передача данных от накопителя ТНЗ реализовано по сетевому протоколу IEEE 802.3 Ipv4 UDP. Сетевой интерфейс ПК для взаимодействия с блоком ТНЗ должен быть настроен в соответствии с настройкой Ethernet блока ТНЗ.

Настройки канала Ethernet блока ТНЗ по умолчанию:

- MAC адрес: 255.255.255.255;
- IP адрес ТНЗ: 192.168.0.1;
- IP адрес получателя: 255.255.255.255 (Broadcast);
- UDP-порт данных: 13332;
- UDP-порт команд: 13332.

Изменения значений настроек по умолчанию осуществляются изменением задания накопителя ТНЗ (tcfg-файл) в СПО «ТН Лаб».

Для работы с накопителем ТНЗ в режиме реального времени задание блока ТНЗ должно быть сконфигурировано так, чтобы установленные модули были настроены на регистрацию данных и на передачу данных в реальном времени (выставлен флаг **«Выдавать данные модуля в реальном времени»**, может меняться по команде извне), а также один из портов платы сбора был настроен на работу в режиме реального времени. При этом необходимо учитывать общую усреднённую пропускную способность канала Ethernet 1000Base-TX от накопителя ТНЗ не более 80 Мбайт/сек (разово не более 100 Мбайт/сек).

Более подробно о настройках задания блока ТНЗ указано в главе **«РАБОТА С ТНЗ В РЕЖИМЕ СЧИТЫВАНИЯ ИНФОРМАЦИИ»** руководства по эксплуатации ТНЗ.

3.2. Настройка утилиты «TNParserRT» для захвата UDP-потока накопителя ТНЗ.

Соединение и передача данных от утилиты «TNParserRT» потребителям также реализовано по сетевому протоколу IEEE 802.3 Ipv4 UDP.

Утилита «TNParserRT» захватывает поток с одного накопителя ТНЗ на заданных сетевом интерфейсе (опционально), IP-адресе (опционально) и UDP-порте.

Если для захвата данных требуется явно использовать заданный сетевой интерфейс, необходимо установить стороннюю библиотеку *libpcap* (*winpcap*, *npcap*), а для ОС семейства Linux также необходимо предоставить утилите «TNParserRT» права суперпользователя (например, запустив утилиту через *sudo*).

Для корректного разбора входного UDP-потока от накопителя ТНЗ утилите «TNParserRT» необходим файл актуального задания ТНЗ, соответствующего этому UDP-потoku. Для указания файла задания используется опция **-c, --tcfg**. В качестве файла задания необходимо передать файл задания накопителя ТНЗ в формате **tcfg**. По умолчанию утилита «TNParserRT» использует файл **tn3.tcfg**, находящийся в локальной папке утилиты. Также в качестве задания ТНЗ утилите можно передать файл реальной записи накопителя в форматах **tnd** и **tne** ([см. подробности ниже](#)).

Для формирования выходных UDP-потоков утилиты «TNParserRT» используется файл в формате JSON, который необходимо указать в параметре вызова **-o, --out-conf**. По умолчанию утилита «TNParserRT» использует файл **out.json** в папке утилиты.

По умолчанию при запуске утилита «TNParserRT» принудительно завершает работу всех запущенных до неё экземпляров утилиты. Для захвата потока от двух и более накопителей потребуются запуск соответствующего числа экземпляров утилиты «TNParserRT» со своими настройками захвата и выдачи и с ключом **-K, --no-kill**.

Конфигурирование сетевых параметров захвата UDP-потока накопителя ТНЗ производится через параметры вызова утилиты:

-i, --interface выбрать сетевой интерфейс для захвата с использованием библиотеки *libpcap*;

-a, --ip-address выбрать IP-адрес для захвата;

-p, --port выбрать UDP-порт для захвата.

Пример захвата потока ТНЗ на UDP-порте 13332:

```
tnparserrt -p 13332
```

Пример захвата потока ТНЗ на интерфейсе *eth0*, IP-адресе 192.168.0.5 и UDP-порте 13332:

```
tnparserrt -i eth0 -a 192.168.0.5 -p 13332
```

Доступные сетевые интерфейсы можно посмотреть вызовом утилиты с ключом **-D** (**--list-interfaces**):

```
tnparserrrt -D
```

Ключ вызова **-C** (**--print-tcfg**) выводит в консоль конфигурацию распределенной системы распарсенного *tcfg*-файла задания ТНЗ:

```
tnparserrrt -c test.tcfg -C
```

Ключ вызова **-O** (**--print-out-conf**) выводит в консоль конфигурацию выходных потоков данных из файла *out.json*:

```
tnparserrrt -c test.tcfg -o out.json -O
```

Ключ вызова **-R**, **--non-rt-slots** принудительно включает разбор данных во входном потоке тех модулей, которые в *tcfg*-файле сконфигурированы только на регистрацию данных во внутреннем хранилище без выдачи в реальном режиме времени в UDP.

Ключ вызова **-r**, **--record-modules** удаляет в конфигурации парсеров, прочитанной из *tnd/tne*-файла, те модули, которые были включены в исходной конфигурации ТНЗ, но не были обнаружены платой сбора при записи (при реальной работе накопителя).

Примечание 1. Рекомендуется всегда применять ключ **-R** при отладке работы утилиты *tnparserrrt* совместно с утилитой *tnreplay* при проигрывании *tnd/tne*-записей.

Примечание 2. Рекомендуется также при проигрывании *tnd/tne*-записей утилитой *tnreplay* в качестве конфигурационного файла утилиты *tnparserrrt* указывать [сам файл записи](#) и ключ **-r** (отключение необнаруженных модулей).

Примечание 3. Рекомендуется никогда не использовать ключ **-R** при отладке работы утилиты *tnparserrrt* совместно с утилитой *tcpreplay* при проигрывании *pcap*-записей или при работе на реальном UDP-потоке от накопителя ТНЗ.

Ключи вызова утилиты **--en** и **--ru** позволяют принудительно выбрать английский или русский языки соответственно для вывода подсказок.

Все параметры утилиты можно посмотреть вызовом утилиты с ключом **-h (--help)**:

```
tnparserrt -h
```

3.3. Использование файлов записей ТНЗ в качестве файла задания «TNParserRT».

В качестве файла задания ТНЗ утилите можно передать файл реальной записи накопителя в форматах **tnd** и **tne**.

3.4. Конвертирование файлов записей ТНЗ в файл задания «TNParserRT».

Пример чтения конфигурации ТНЗ из файла записи с выводом её на экран на русском языке и экспортом в tcfg-файл с отключением необнаруженных модулей:

```
tnparserrt -c record.tne -o tn3.tcfg -C -r --ru
```

4. Выходной формат выдачи данных утилиты «TNParserRT» в UDP.

Выходной поток утилиты «TNParserRT» на каждом тике состоит из маркерного UDP-пакета со служебной информацией «INFO» и одного или нескольких пакетов данных модулей «DATA», собранных блоком ТНЗ и разобранных утилитой на этом тике (таблица 1).

Таблица 1. Формат полного Ethernet-пакета утилиты «TNParserRT».

Ethernet header (802.3)	IPv4 Header (802.3)	UDP Header (802.3)	UDP Data (payload) «INFO» или «DATA»	Checksum (802.3)
14 байт	20 байт	8 байт	raw bytes	2 байта

Маркерный пакет «INFO» и пакеты данных модулей «DATA» составляют один информационный «фрейм» данных утилиты «TNParserRT» на данном тике накопителя ТНЗ.

Все поля маркерного пакета «INFO» и пакетов данных «DATA» отправляются без промежутков (выравнивающих байт) младшими байтами вперёд.

Если размер итогового UDP-пакета данных «DATA» превышает 1024 байт, он разбивается на UDP-пакеты размером 1024 байт. Последний пакет при этом может быть меньше 1024 байт.

4.1. Формат маркерного UDP-пакета «INFO».

Поля данных маркерного пакета данных утилиты «TNParserRT» представлены в таблице 2.

Таблица 2. Маркерный пакет «INFO» утилиты «TNParserRT».

№ поля	Название	Размер, байт	Тип	Значение (пример)	Комментарий к значению в примере
1	Marker	2	uint16	0x4e54	В ASCII "TN"
2	DataCount	2	uint16	3	Будут отправлены 3 набора данных "DATA"
3	Counter	4	uint32	125	Номер отправляемого фрейма
4	Data	DataCount*2	uint16	4	Будет отправлен один пакет с 4 байтами данных

№ поля	Название	Размер, байт	Тип	Значение (пример)	Комментарий к значению в примере
	counters				канала
5			uint16	2000	Будут опрарвлены 2 пакета данных – 1024 и 976 байт
6			uint16	4096	Будут опрарвлены 4 пакета данных – по 1024 байт
7	Checksum	2	uint16	0x9958	Включая маркер, не включая Checksum и последующие поля
8	Temperature	2	int16	360	22.5°C. Градусы Цельсия получаются делением на 16.
9	резерв	30	char	-	резерв/служебная информация

Описание полей маркерного пакета утилиты:

1. Marker – признак начала маркерного пакета, имеет значение 0x4e54 (в ASCII “TN”);
2. DataCount — число счётчиков данных «DATA», передаваемых в текущем фрейме после маркерного пакета.
3. Counter – счётчик отправленных фреймов;
4. Data Counters – счётчики данных. Каждый счётчик имеет размерность 2 байта, и указывает количество байт, которое будет передано в последующих UDP-пакетах для каждого набора данных;
5. Checksum – контрольная сумма маркерного пакета, включая маркер и исключая саму контрольную сумму, а также все служебные данные, следующие после контрольной суммы;
6. Temperature – поле служебных данных утилиты «TNParserRT». На данный момент выдаётся температура мастер-блока ТНЗ. Поля служебных данных пока не зафиксированы и могут быть изменены в дальнейшем.

4.2. Алгоритм формирования контрольной суммы маркерного пакета.

Поле Checksum содержит дополнение до «0» суммы всех байтов UDP-пакета, начиная от нулевого байта поля маркера и исключая саму контрольную сумму и все служебные данные, следующие после Checksum, рассматриваемых как 2-х байтные слова, без перехода бита переполнения в младший разряд.

Функция расчёта контрольной суммы на языке «C++»

```
uint16 checksum(const void *buffer, int size, uint16 sum)
{
    auto *ptr = (const uint16*)buffer;
    while (size > 0)
    {
        sum += *ptr;
        size -= 2;
        ptr++;
    }
    return uint16(uint32(0x10000) - sum);
}
```

4.3. Режим формирования выходного UDP-потока без маркерного пакета.

В конфигурации выходного потока (в формате JSON) имеется необязательный параметр "marker" ([см. таблицу 4](#)). Явное указание значение "no" для этого параметра отключает формирование и выдачу маркерного пакета «INFO» фрейма данных для соответствующего выходного потока. При этом утилита «TNParserRT» формирует и выдаёт потребителю только UDP-пакеты данных «DATA».

Это позволяет на приёмной стороне UDP-потока от утилиты «TNParserRT» подать разобранные данные модулей сразу на отображение.

4.4. Подмешивание номеров тиков ТНЗ в UDP-пакеты данных «DATA».

Явное указание значения «yes» для необязательного параметра «ticks» в параметрах выходного потока включает подмешивание номеров 1024-герцовых тиков накопителя ТНЗ в выходные UDP-пакеты данных «DATA» утилиты «TNParserRT».

Данный режим может быть полезен при необходимости синхронизации по номеру тика накопителя различных выходных потоков утилиты «TNParserRT» от различных модулей распределенной системы.

При включении этого режима выходной UDP-пакет данных «DATA» утилиты разбивается на «пачки» данных, соответствующие отдельному тiku накопителя. Перед каждой такой пачкой данных на тике в UDP-пакет дополнительно вставляются четыре байта (uint32) номера тика ТНЗ и два байта (uint16) размера этой пачки в байтах.

Пример. Допустим мы сконфигурировали выходной поток утилиты «TNParserRT» на выдачу полного сырого потока данных одного канала АЦП/301 каждые два тика ТНЗ. Частота дискретизации АЦП/301 выбрана 131072 Гц.

При этом в один выходной UDP-пакет «DATA» будут упакованы две пачки данных, состоящие из ста двадцати восьми 16-битных измерения АЦП (см. таблицу 3).

Таблица 3. UDP-пакет «DATA» с подмешанными тиками ТНЗ.

UDP-пакет данных «DATA»						
	№ тика	Размер	Пачка данных	№ тика	Размер	Пачка данных
тип	uint32	uint16	raw bytes	uint32	uint16	raw bytes
hex	01 00 00 00	00 01	256 байт	02 00 00 00	00 01	256 байт
dec	1	256	128 x int16	2	256	128 x int16

Примечание 1. Для корректной синхронизации нумерации тиков дочерних кожухов распределенной системы с мастер-блоком необходимо в конфигурации ТНЗ (*tcfg*-файл) для дочерних блоков указывать «Абсолютное время» в разделе «Синхронизация времени». Для более подробной информации о настройке синхронизации времени в распределённой системе необходимо обратиться к руководству по эксплуатации накопителя ТНЗ.

Примечание 2. В этом режиме рекомендуется выбирать частоту выдачи выходного потока таким образом, чтобы размер данных результирующего UDP-пакета данных «DATA» (вместе с подмешанными тиками) не превышал 1024 байт. В противном случае этот пакет будет разбит на несколько UDP-пакетов с размером данных, не превышающим 1024 байт. Это может затруднить разбор данных на приёмной стороне. А в случае потерь приёмником промежуточных UDP-пакетов данных и невозможность их корректного разбора.

5. Конфигурирование выходных потоков утилиты «TNParserRT».

Конфигурирование выходного потока СПО «TNParserRT» осуществляется с помощью конфигурационного файла в формате JSON.

Для каждого из выходных потоков можно задать следующие параметры выдачи в зависимости от типа (режима) потока:

Таблица 4. Параметры выходного потока.

db	имя или полный путь к базе данных в режиме SQL
host	IP-адрес назначения выходного UDP-потока или хоста БД PostgreSQL. Для широковещательной выдачи — задать адрес 255.255.255.255 или broadcast
port	UDP-порт назначения выходного UDP-потока или доступа к БД PostgreSQL
tickcount	делитель частоты 1024-герцового тика ТНЗ для выдачи данных (имеет приоритет перед параметром timeout)
timeout	таймаут периодической выдачи данных в миллисекундах
mode	режим выдачи данных на весь поток (таблица 6). По умолчанию «raw»
marker	значение «no» - отключает отправку маркерного кадра. Значение по умолчанию - «yes»
ticks	значение «yes» - включает подмешивание номеров тиков ТНЗ в UDP-пакеты данных . Значение по умолчанию - «no»
sql	тип базы данных для режима SQL. Если не указан, выбирается режим UDP
slots	массив выдаваемых сигналов с разделением данных до канала (параметра) модуля отдельного блока (кожуха). (таблица 4)
tables	массив таблиц БД для режима SQL

Для каждого выходного слота выходного потока можно задать следующие параметры (таблица 5):

Таблица 5. Параметры отдельного слота выходного потока.

dataCol	имя столбца таблицы для сохранения параметра в режиме SQL
mode	режим выдачи данных слота (таблица 6). Если явно не задан, будет взят из режима, заданного для всего выходного потока
block, cover	номер блока (кожуха) в распределенной системе для выходного потока
slot	номер модуля (слота) в выбранном блоке для выходного потока
channel, line	список выдаваемых каналов выбранного модуля для выходного потока
param	список выдаваемых параметров выбранных каналов модуля

Таблица 6. Режимы выдачи данных выходного потока и слотов.

Режим	Тип данных	Описание
raw	int16, uint16, uint32	Выдача последних полученных измерений в «сыром» виде (цифра) по таймауту или делителю тика так, как их выдаёт соответствующий модуль
real	float	Выдача последних полученных измерений в физических величинах там, где это возможно. Например, напряжения или сопротивления для АЦП
all	char, int16, uint16, uint32	Выдача всех измерений, накопленных за период выдачи в «сыром» виде (цифра) так, как их выдаёт соответствующий модуль

Режим	Тип данных	Описание
all-real	float	Выдача всех измерений, накопленных за период выдачи в физических величинах там, где это возможно. Например, напряжения или сопротивления для АЦП
nmea	ASCII	Выдача текстовых сообщений в формате NMEA для модулей СНС
udp	char	Выдача данных UDP-пакета (payload) для сетевых модулей МПК и ПК платы сбора

5.1. Нумерация кожухов, слотов, каналов и параметров в конфигурационном файле.

Нумерация кожухов в JSON-файле выходных потоков осуществляется в следующем порядке.

Нумерация кожухов в конфигурационном JSON-файле для распределенной системы начинается от нуля.

Мастер-блок имеет номер «0».

Дочерние кожухи нумеруются последовательно, начиная с номера «1», по сетевым разъёмам мастер-блока справа-налево, сверху-вниз, если смотреть на диаграмму мастер-блока в СПО «ТН Лаб». Или сверху вниз, если смотреть на дерево приборов в СПО «ТН Лаб».

Нумеруются все дочерние кожухи, подключённые к мастер-блоку, даже если они не сконфигурированы на регистрацию или выдачу данных в real-time.

Таким образом, в самой максимальной конфигурации распределённой системы, когда к мастер-блоку подключены 22 дочерних блока (2 к разъёмам ПК-2 и 20 — через модули МПК) будет следующая нумерация:

- номер «0» - мастер-блок;
- номер «1» - дочерний блок, подключенный к разъёму X4 модуля ТНЗ.ПК-2;

- номер «2» - дочерний блок, подключенный к разъёму X5 модуля ТНЗ.ПК-2;
- номер «3» - дочерний блок, подключенный к верхнему разъёму модуля МПК/001 в первом слоту;
- номер «4» - дочерний блок, подключенный к нижнему разъёму модуля МПК/001 в первом слоту;
- номер «5» - дочерний блок, подключенный к верхнему разъёму модуля МПК/001 во втором слоту;
- номер «6» - дочерний блок, подключенный к нижнему разъёму модуля МПК/001 во втором слоту;
- ...
- номер «21» - дочерний блок, подключенный к верхнему разъёму модуля МПК/001 в десятом слоту;
- номер «22» - дочерний блок, подключенный к нижнему разъёму модуля МПК/001 в десятом слоту.

Нумерация слотов, каналов и параметров в конфигурационном JSON-файле начинается от единицы (а не от нуля, как в массивах на языке «С»). Такая же нумерация (от единицы) слотов и каналов используется в СПО «ТН Лаб» при составлении задания накопителя ТНЗ.

5.2. Информация о статусах кожухов ТНЗ.

Также в поле "slot" выходного потока слота отдельного кожуха можно через запятую указать выдачу статусных данных этого кожуха (таблица 7). При этом остальные поля для этого потока необязательны и игнорируются.

Таблица 7. Статусные данные выходного потока кожуха.

Режим	Тип данных, размер	Описание
status	uint16, 2 байта	статусное слово накопителя ТНЗ (таблица 8)
error	uint32, 4 байта	ошибки разбора данных кожуха (таблица 9)

Режим	Тип данных, размер	Описание
temp	Int16, 2 байта	температура кожуха в кодах; Градусы Цельсия получаются делением на 16
ftemp	float, 4 байта	температура кожуха в градусах Цельсия
tick	uint32, 4 байта	метка времени кожуха (номер тика)
frame	uint32, 4 байта	счётчик полученных кадров данных кожуха

Таблица 8. Значения битов статусного слова ТНЗ.

№ бита	Значение
0	Флаг режима работы: 0 — нормальный режим; 1 — установлен режим ограниченной информативности.
1-10	Флаги технической информации (ошибок). Каждый бит, соответствует определённому слоту в блоке и состоянию. 0 — нет ошибки; 1 — ошибка (отсутствие синхронизации или отказ модуля в слоте N, где N - номер бита). Является ошибкой если плата в данном слоте физически установлена.
11-15	Зарезервированы, не используются.

Таблица 9. Коды ошибок парсера данных кожуха.

0x0	нет ошибок;
0x1	tcfg-файл не соответствует входному UDP-потoku
0x2	json-файл не соответствует tcfg-файлу
0x3	был потерян хотя бы один фрейм с начала запуска утилиты

0x4	за последние 10 секунд был потерян хотя бы один фрейм
-----	---

5.3. Конфигурирование выходных потоков для модулей АЦП.

Файл конфигурации выходного потока СПО «TNParserRT» задаётся в формате JSON-массива выходных потоков на заданные IP-адреса и порты.

Пример:

```
[{
  "state":      "on",          // включить
  "host":       "127.0.0.1",   // IP-адрес приёмника данных
  "port":       "50001",       // UDP-порт приёмника данных
  "timeout":    "100",         // 100мс - частота 10Гц
  "slots":
  [
    {
      "block": "0",           // выдавать данные с основного кожуха (0 - мастер)
      "slot":  "temp,error,tick", // выдавать статусные данные кожуха
    },{
      "mode":    "raw", // выдавать сырые значения в кодах АЦП
      "block":   "0",   // выдавать данные с основного кожуха (0 - мастер)
      "slot":    "1",   // выдавать данные от АЦП из первого слота
      "channel": "1-5"  // выдавать данные с 1,2,3,4 и 5-го каналов АЦП
    }
  ]
},{
  "state":      "on",          // включить выдачу
  "host":       "127.0.0.1",   // IP-адрес приёмника данных
  "port":       "50002",       // UDP-порт приёмника данных
  "tickcount":  "100",         // выдавать каждые 100 тиков ТНЗ
  "slots":
  [
    {
      "mode":    "real",      // выдавать значения в вольтах (float)
      "block":   "1",         // выдавать данные с дочернего кожуха (1-й дочерний)
      "slot":    "2",         // выдавать данные от АЦП из второго слота
      "channel": "2,7"        // выдавать данные со 2 и 7-го каналов АЦП
    }
  ]
},{
  "state":      "on",          // включить выдачу
  "host":       "127.0.0.1",   // IP-адрес приёмника данных
  "port":       "50003",       // UDP-порт приёмника данных
  "tickcount":  "10",          // выдавать весь набор данных за 10 тиков ТНЗ
  "slots":
  [
    {
      "mode":    "all",       // выдавать весь поток канала слота
      "block":   "0",         // выдавать данные с основного кожуха
      "slot":    "1",         // выдавать данные от АЦП из первого слота
      "channel": "1"          // выдавать все данные с 1-го канала АЦП
    }
  ]
},{
  "state":      "on",          // включить выдачу
  "host":       "127.0.0.1",   // IP-адрес приёмника данных
```

```

"port":      "50004",          // UDP-порт приёмника данных
"tickcount": "10",            // выдавать весь набор данных за 10 тиков ТНЗ
"slots":
[
  {
    "mode": "all-real",        // выдавать весь поток канала слота в вольтах
    "cover": "0",              // выдавать данные с мастер-кожуха
    "slot": "1",               // выдавать данные от АЦП из первого слота
    "line": "1"                // выдавать с 1-го канала АЦП
  }
]
}]

```

В этом примере для четырёх выходных потоков реализованы различные способы накопления и выдачи данных, полученных от ТНЗ.

Первый выходной поток выдаётся на адрес 127.0.0.1, порт 50001 каждые 100мс. В поле `slots` задана выдача пяти каналов АЦП из первого модуля(слота) мастер блока. При этом будет выдано одно последнее измерение по каждому каналу слота, полученное на момент выдачи выходного потока.

Второй выходной поток выдаётся на адрес 127.0.0.1, порт 50002. Выдача осуществляется на каждый сотый 1024-герцовый «тик» выдачи данных накопителем ТНЗ. Выданы будут данные с 2-го и 7-го каналов второго модуля первого дочернего блока. При этом будет выдано одно последнее измерение по каждому каналу модуля, полученное на момент выдачи выходного потока. Данные АЦП будут выданы в физических величинах в соответствии с коэффициентом, заданном в `tcfg`-задании для выбранных каналов модуля.

Третий выходной поток выдаётся на адрес 127.0.0.1, порт 50003. Выдача осуществляется на каждый десятый «тик» выдачи данных накопителем ТНЗ. Данные будут выданы с 1-го канала 1-го слота мастер-блока, накопленные за всё время с момента последней выдачи потребителю.

Четвёртый выходной поток выдаётся на адрес 127.0.0.1, порт 50004. Выдача осуществляется на каждый десятый «тик» выдачи данных накопителем ТНЗ. Данные будут выданы с 1-го канала 1-го слота мастер-блока, накопленные за всё время с момента последней выдачи потребителю в соответствующих физических величинах АЦП (вольтах, омах итд).

Примечание 1. Для выходного потока параметр `tickcount` является приоритетным по отношению к `timeout`.

Примечание 2. Если для выходного потока задан режим «all» или «all-real», то для формирования выходного потока будут использованы данные только одного, первого

канала, первого слота блока, указанных в поле «*slots*». Остальные каналы выданы не будут. Так как невозможно будет разобрать на приёмной стороне смешанные данные с нескольких каналов, слотов, блоков.

5.3.1 Модули АЦП/010 и АЦП/310.

Для АЦП/010 и АЦП/310 для конфигурирования выходного потока выбранного канала АЦП в поле «*param*» необходимо также указать необходимые параметры выдачи.

Нумерация параметров канала:

- 1 - мгновенное значение переменного напряжения;
- 2 - среднеквадратическое значение переменного напряжения;
- 3 - частота переменного напряжения.

5.4. Конфигурирование выходных потоков для модулей ARINC-429.

5.4.1 Режим "*all*".

В режиме "*all*" выдаются все слова (метки) ARINC-429, накопленные за период выдачи в виде 32-битных слов одного канала.

Пример:

```
{
  "state":      "on",           // включить выдачу
  "host":       "127.0.0.1",    // IP-адрес приёмника данных
  "port":       "50001",        // UDP-порт приёмника данных
  "timeout":    "100",          // 100мс - частота 10Гц
  "slots":     [
    {
      "mode":    "all",         // выдавать все метки канала ARINC-429, файловый приём
      "block":   "0",           // берём данные с основного кожуха
      "slot":    "1",           // берём данные от ТН3/A429/001 из первого слота
      "channel": "3",           // берём данные с третьего канала ARINC-429 модуля
    }
  ]
}
```

5.4.1 Периодический режим *"raw"*.

Периодический режим по таймауту или делителю тика в режиме *"raw"* является адресным. Парсер имеет буфер на 256 меток для каждого канала ARINC-429, полученных на момент выдачи. И в момент выдачи выдаёт только указанные в конфигурации метки.

Пример:

```
{
  "state":      "on",          // включить выдачу
  "host":       "127.0.0.1",   // IP-адрес приёмника данных
  "port":       "50001",       // UDP-порт приёмника данных
  "timeout":    "100",         // 100мс - частота 10Гц
  "slots":
  [
    {
      "mode":     "raw",        // выдавать выбранные метки ARINC-429, адресный приём
      "block":    "0",          // берём данные с основного кожуха
      "slot":     "1",          // берём данные от TH3/A429/001 из первого слота
      "channel":  "3",          // берём данные с третьего канала
      "param":    "0100, 0110, 0203, 0205-0210" // выдаём только выбранные метки,
                                                    // последние полученные на момент выдачи
                                                    // в виде 32-битных слов
    }
  ]
}
```

Набор выдаваемых меток линии ARINC-429 в поле *«param»* должен быть задан в явном формате языка "C":

восьмеричном, начиная с нуля: 075, 0103-0310,

шестнадцатеричном, начиная с 0x: 0x10, 0x0A-0x10,

или десятичном.

5.4.1 Периодический режим *"real"*.

В этом режиме для преобразования слова в физическую величину необходимо задать дополнительные параметры (таблица 10).

Таблица 10. Конфигурация выдачи физических значений из слов ARINC-429.

msb	номер старшего разряда данных от 1
lsb	номер младшего разряда данных от 1

price	цена младшего разряда
sdi	матрица источника/приёмника данных в двоичном виде («no» - не использовать)
sign	признак наличия знака

При этом в UDP будет выдано знаковое или беззнаковое значение, заложенное в разрядах *lsb...msb* слова ARINC-429, умноженное на «*price*» с учётом матрицы *SDI*.

ВАЖНО! Для знаковых значений в поле «*msb*» указывается номер бита знака. Знаковое целое, заложенное в разрядах *lsb...msb* слова ARINC-429 интерпретируется как знаковое целое в дополнительном коде. И оно умножается на значение ЦМР в поле «*price*».

Пример:

```
{
  "state":      "on",           // включить выдачу
  "host":       "127.0.0.1",    // IP-адрес приёмника данных
  "port":       "14442",        // UDP-порт приёмника данных
  "timeout":    "40",           // 40мс - частота выдачи 25Гц
  "mode":       "real",         // все данные выдавать в физ. величинах
  // "marker":  "off",          // по-умолчанию - выдача маркерного UDP-пакета
  "slots": [
    {
      "block":   "9",
      "slot":    "3",
      "channel": "13",
      "param":   "005",         // адрес слова A429
      "msb":     "28",          // номер старшего разряда
      "lsb":     "19",          // номер младшего разряда
      "price":   "0.25",        // цена младшего разряда
      "sdi":     "no",          // игнорировать SDI, выдавать все слова с адресом 005
      "sign":    "yes"          // в данных знаковое значение
    }, {
      "block":   "9",
      "slot":    "2",
      "channel": "12",
      "param":   "0103",
      "msb":     "28",
      "lsb":     "20",
      "price":   "0.25",
      "sdi":     "00",          // выдавать слова с адресом 103 и SDI 00
      "sign":    "no"          // в данных беззнаковое значение
    }, {
      "block":   "9",
```

```

        "slot":      "2",
        "channel":   "10",
        "param":     "071",
        "msb":       "28",
        "lsb":       "18",
        "price":     "0.25",
        "sdi":       "10",          // выдавать слова с адресом 071 и SDI 10
        "sign":      "no"
    } ]
}

```

5.4.1 JSON-секция "A429"

Конфигурационный JSON-файл может содержать секцию "A429", предназначенную для однократного указания коэффициентов пересчёта слов ARINC429 в физические величины для всех выходных потоков этого файла.

Пример:

```

...
// один раз приведём все пересчёты в «физику» для всех блоков, слотов, линий и меток ARINC429
"A429":[
    // 1. Напряжение на аварийной шине
    {"block":"9","slot":"2","line":"12","param":"0103","sdi":"00","sign":"no","msb":"28","lsb":"20","price":"0.25"},
    // 2. Частота шины наземного источника
    {"block":"9","slot":"2","line":"10","param":"071","sdi":"10","sign":"no","msb":"28","lsb":"18","price":"0.25"},
    // 3. Обороты ВСУ
    {"block":"9","slot":"3","line":"13","param":"005","sdi":"no","sign":"no","msb":"28","lsb":"19","price":"0.25"},
    ...
]}
...

```

И в конфигурации потоков указываем только:

```

...
"slots":[
    // 1. Напряжение на аварийной шине
    { "block": "9", "slot": "2", "line": "12", "param": "0103", "sdi": "00", "mode": "real" },
    // 2. Частота шины наземного источника
    { "block": "9", "slot": "2", "line": "10", "param": "071", "sdi": "10", "mode": "real" },
    // 3. Целевые Обороты ВСУ
    { "block": "9", "slot": "3", "line": "13", "param": "005", "sdi": "no", "mode": "real" },
    ...
]

```

5.5. Конфигурирование выходных потоков для модулей CAN/RS.

Для каналов последовательных портов предусмотрен только режим "all" - выдаётся весь «сырой» поток байтов, накопленных за период выдачи.

Для каналов CAN режим "all" — выдаётся весь поток упакованных CAN-пакетов с одного канала на UDP-порт в виде ID-LEN-DATA (поточковый приём).

Вероятно, будет нюанс с растущим буфером накопления.

Для каналов CAN режим *"raw"* - периодическая выдача упакованных CAN-пакетов, выбранных по ID, с одного канала в виде ID-LEN-DATA (адресный приём).

5.6. Конфигурирование выходных потоков для модулей IRIG.

Для каналов последовательных портов предусмотрен только режим *"all"* - выдаётся весь «сырой» поток байтов, накопленных за период выдачи.

Для каналов IRIG на данный момент предусмотрен только режим *"raw"* — выдача «сырого» 16-битного целого в заданном миноре и смещении. Для этого кроме задания номера линии необходимо также указать номер подзадания, номер минора и смещение на необходимую ячейку мажора.

Пример:

```
{
  "state":      "on",           // включить выдачу
  "host":       "127.0.0.1",    // IP-адрес приёмника данных
  "port":       "14442",        // UDP-порт приёмника данных
  "timeout":    "40",           // 40мс - частота выдачи 25Гц
  "slots": [
    {
      "mode":     "raw",         // выдавать «сырые» 16-битные целые
      "block":    "9",
      "slot":     "3",
      "line":     "1",           // линия IRIG №1. применён синоним поля «channel»
      "task":     "1",           // номер подзадания
      "minor":    "3",           // номер минора в мажоре от единицы
      "offset_num": "4",         // номер смещения 16-битной ячейки данных в миноре от
единицы
    }, {
      "mode":     "raw",         // выдавать «сырые» 16-битные целые
      "block":    "9",
      "slot":     "3",
      "line":     "1",           // линия IRIG №1.
      "task":     "1",           // номер подзадания
      "minor":    "3",           // номер минора в мажоре
      "offset_bits": "48",       // смещение 4-й 16-битной ячейки данных в миноре в битах
    }
  ]
}
```

Скорее всего впоследствии будет реализован режим *«all»* - выдача всего сырого потока мажоров IRIG для заданного подзадания заданной линии.

5.7. Конфигурирование выходных потоков для модулей ТНЗ/СНС/001.

Для модулей СНС реализованы два режима:

"all" — выдаётся весь поток байт, полученный с последовательного порта спутникового приёмника за период выдачи;

"nmea" — выдаются только полученные за период выдачи текстовые сообщения в формате NMEA.

5.8. Конфигурирование выходных потоков для модулей ТНЗ/МПК/301 и ТНЗ.ПК-1/2.

Для модулей ТНЗ/МПК/301 и ТНЗ.ПК-1/2 на плате сбора предусмотрены режимы "all" и "udp".

В режиме "all" выдаётся весь «сырой» поток байт, полученный по соответствующему каналу Ethernet, включая заголовки Ethernet, IP, UDP и данные.

В режиме "udp" выдаются выделенные из Ethernet-пакета данные UDP-пакета (payload).

5.9. Конфигурирование выходных потоков для модулей ТНЗ/СЧ/301.

В периодическом режиме по таймауту или делителю тика в режиме "raw" выдаются выбранные параметры каналов (частоты, периоды или длительности).

Нумерация параметров канала:

1. частота ПФ, количество переходов Кн→Кв (ПФ) за период измерения;
2. частота ОФ, количество переходов Кв→Кн (ОФ) за период измерения;
3. период ПФ, время между переходами Кн→Кв (ПФ) и Кн→Кв (ПФ);
4. период ОФ, время между переходами Кв→Кн (ОФ) и Кв→Кн (ОФ);
5. длительность ПИ, время между переходами Кн→Кв (ПФ) и Кв→Кн (ОФ);
6. длительность ОИ, время между переходами Кв→Кн (ОФ) и Кн→Кв (ПФ).

Пример:

```
{  
  "state":      "on",           // включить выдачу  
  "host":       "127.0.0.1",    // IP-адрес приёмника данных
```

```

"port":      "50001",      // UDP-порт приёмника данных
"timeout":   "100",       // 100мс - частота 10Гц
"slots": [
{
    "block":    "0",       // выдавать данные с основного кожуха
    "slot":     "1",       // выдавать данные от TH3/A429/001 из первого слота
    "channel":  "3",       // выдавать данные с третьего канала
    "param":    "1,4-6"    // выдаём только выбранные параметры,
                          // последние полученные на момент выдачи
                          // в виде 32-битных слов -
                          // частота ПФ, период ОФ, длительности ПИ и ОИ.
} ]
}

```

5.10. Конфигурирование выдачи в базу данных SQLITE.

Сохранение данных в базу SQLITE производится периодически по таймауту или делителю тика. В режимах "raw" или "real" сохраняются выбранные параметры каналов модулей (напряжения, частоты, периоды, длительности итд) в заданные столбцы заданной строки заданной таблицы БД SQLITE. Для сохранения данных в одну и ту же строку таблицы для выходного потока задаются имя и значение первичного ключа в полях «rowidCol» и «rowidValue». Для сохранения метки времени обновления данных в строке задаётся имя соответствующего столбца в поле «timeCol». Метка времени сохраняется в виде числа миллисекунд, прошедших с полуночи (00:00:00 UTC) 1 января 1970 года (эпоха Unix).

За создание базы данных и соответствующих таблиц с заданными столбцами отвечает заказчик. СПО «TNParserRT» осуществляет только обновление в имеющихся таблицах существующей БД.

Пример выходного потока:

```

{
    "state":      "on",      // включить
    "tickcount":  "100",     // выдача по делителю тика
    "sql":        "sqlite",  // тип SQL-базы
    "db":         "db.sqlite", // полный или относительный путь к БД sqlite
    "tables": [ // массив таблиц для сохранения данных
    {
        "table":    "data",  // имя таблицы
        "rowidCol": "rowid", // имя столбца первичного ключа
        "rowidValue": "7",    // значение первичного ключа
        "timeCol":   "ts",    // имя столбца метки времени
        "slots": [ // массив столбцов таблицы для записи данных
        {
            "dataCol": "adc3_real", // имя столбца для сохранения
            "mode":     "real",      // тип данных

```

```

        "block":    "0",           // номер блока
        "slot":     "1",           // номер слота
        "channel":  "3",           // номер линии
        "param":    "6",           // номер параметра
    }, {
        // следующий столбец таблицы "data"
    } ]
}, {
    // следующая таблицы
} ]
}

```

Примечание: При выборе частоты обновления значений в таблицах в режиме SQL следует учитывать время, необходимое для выполнения транзакций сохранения данных в БД. Если выполнения транзакций будет ощутимо большим, СПО «TNParserRT» не гарантирует успешный приём и разбор полного входного потока от накопителя ТНЗ (возможны пропуски и потери входных данных).

5.11. Конфигурирование выдачи в базу данных PostgreSQL.

Конфигурирование сохранения в БД PostgreSQL в общем аналогично БД SQLITE. Для БД PostgreSQL в выходном потоке необходимо дополнительно указать хост, на котором расположена БД, порт для доступа к БД, имя пользователя базы и пароль:

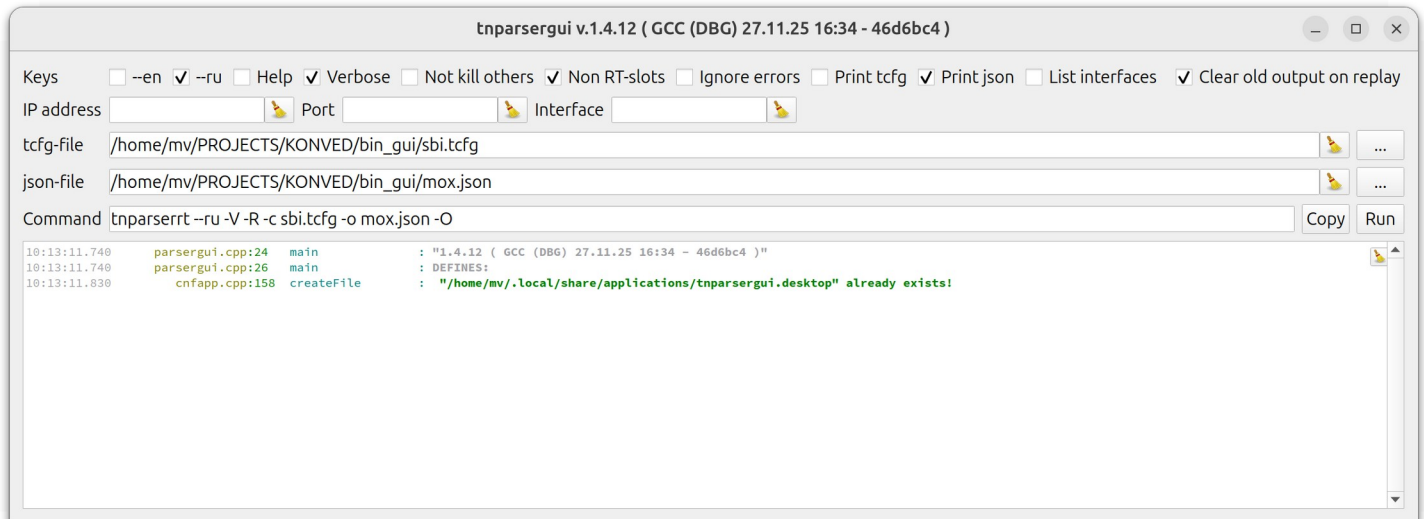
```

{
    "state":        "on",
    "tickcount":    "100",        // частота обновления данных, каждые 100 тиков
    "sql":          "psql",       // тип sql-базы
    "db":           "test_db",    // имя sql-базы
    "host":         "localhost",  // хост расположения БД
    "port":         "5432",       // порт
    "user":         "leo",        // пользователь БД
    "password":     "root12",     // пароль пользователя
    "tables": [ // дальнейшая конфигурация аналогична БД SQLITE.
        ...
    ]
}

```

6. ЧМИ приложение «tnparsergui».

Приложение «tnparsergui» позволяет с помощью удобного графического интерфейса сформировать параметры командной строки для вызова утилиты «tnparserrt» и запустить на выполнение выбранную команду.



7. Утилита «tnreplay».

7.1. Назначение

Утилита «tnreplay» предназначена для воспроизведения записей накопителей ТНЗ с выдачей UDP-потока на заданные IP-адреса и порты.

7.2. Возможности

- воспроизведение ТНЗ-записей форматов *tne* и *tnd*;
- циклическое воспроизведение записей;
- выдача UDP-потока записи на один или несколько заданных IP-адресов и портов;
- воспроизведение записи в реальном времени, с ускорением или замедлением;
- останов воспроизведения файла после выдачи заданного количества UDP-пакетов;
- постановка воспроизведения на паузу после каждого отправленного кадра записи;
- вывод в консоль процента воспроизведения проигрываемой записи;
- ускоренная прокрутка начала воспроизведения на заданное время, процент или тик записи.

7.3. Примеры применения

```
tnreplay -x 5 -p file.tne 127.0.0.1:13332
```

проигрывает файл «*file.tne*» с ускорением 5, с отображением прогресса воспроизведения в консоли и выдачей UDP-потока ТНЗ на адрес 127.0.0.1, порт 13332.

```
tnreplay -p -s 1:20:10.5 file.tne
```

проигрывает файл «*file.tne*», с отображением прогресса воспроизведения, начиная с момента один час двадцать минут и десять с половиной секунд от начала записи, в адрес 127.0.0.1 и порт 13332 по умолчанию.

Все параметры утилиты можно посмотреть вызвав утилиту в терминале с ключом **-h**:

```
tnreplay -h
```

Ключи вызова утилиты **--en** и **--ru** позволяют принудительно выбрать английский и русский языки соответственно для вывода подсказок.

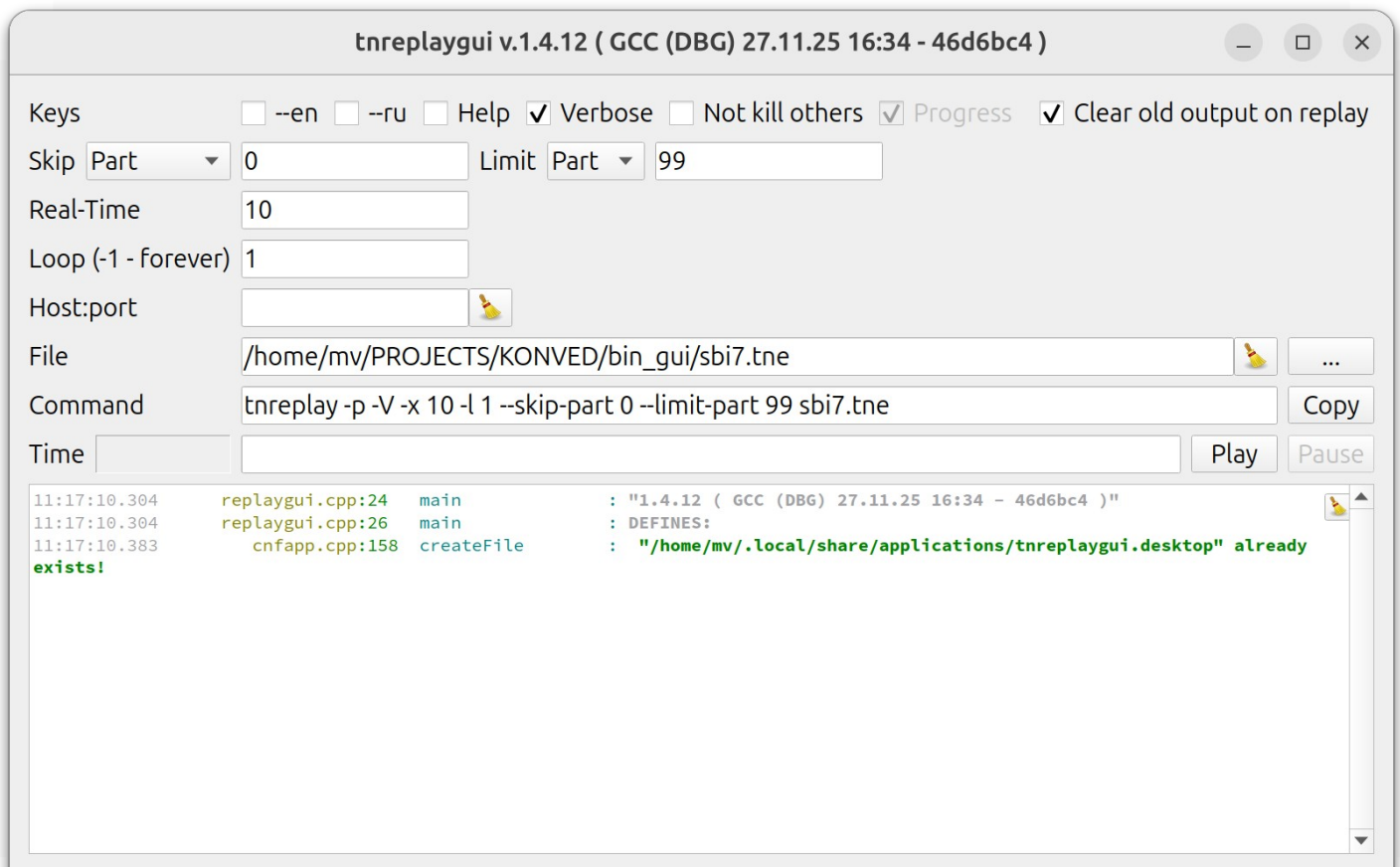
ВАЖНО! Утилита **tnreplay** при воспроизведении tnd/tne-файлов выдаёт в UDP пакеты данных всех модулей системы, для которых в tcfg-файле конфигурации была разрешена регистрация данных, включая модули с **ВЫКЛЮЧЕННОЙ** выдачей данных в реал-тайм в UDP. В то же время утилита **tnparseerrt** штатно разбирает данные только тех модулей, у которых в конфигурации была включена не только регистрация данных, но и **ВЫДАЧА данных в реал-тайм в UDP**.

Для корректной работы связки **tnreplay** + **tnparseerrt** рекомендуется всегда применять ключи **-R** (принудительный разбор данных всех регистрируемых модулей) и **-r** (отключение разбора необнаруженных модулей) утилиты **tnparseerrt**.

По умолчанию при запуске утилита **tnreplay** принудительно завершает работу всех запущенных до неё экземпляров утилиты. При необходимости запуска нескольких экземпляров утилиты **tnreplay** необходимо применять ключ вызова **-K, --no-kill**.

8. ЧМИ приложение «tnreplaygui».

Приложение «*tnreplaygui*» позволяет с помощью удобного графического интерфейса сформировать параметры командной строки для вызова утилиты «*tnreplay*» и запустить на выполнение выбранную команду.



9. Русификация консольного вывода утилит «tnparserrt» и «tnreplay».

Утилиты «*tnparserrt*» и «*tnreplay*» позволяют выводить подсказки на английском или русском языках.

По умолчанию подсказки в утилитах выводятся в соответствии с языком системной локали. Ключи вызова утилиты **--en** и **--ru** позволяют принудительно выбрать английский или русский язык соответственно для вывода подсказок.

Примечание. Для корректного отображения русских подсказок в консоли в ОС семейства Windows необходимо в региональных настройках выбрать "Русский язык" для программ, не использующих Unicode. А для ОС Windows 7 и младше ещё и в свойствах консоли выбрать шрифт, отличный от точечного.